

Time series I

The following tutorial contains Python examples for solving time series forecasting problems. You should refer to the Slides Week 9 to understand some of the concepts introduced in this tutorial.

The goals for this tutorial are as follows:

1. To provide examples of using different statistical methods from the statsmodels library package.
2. To make a model performance comparison for time series forecasting.

Read the step-by-step instructions below carefully. To execute the code, click on the corresponding cell and press the SHIFT-ENTER keys simultaneously.

Statsmodels is a library that provides classes and functions for the estimation of many different statistical models, as well as for conducting statistical tests, and statistical data exploration.

toolbox:

ETS model: error type (E; additive or multiplicative), a trend type (T; additive or multiplicative, both damped or undamped, or none), and a seasonality type (S; additive or multiplicative or none).

```
from statsmodels.tsa.exponential_smoothing.ets import ETSModel
```

https://www.statsmodels.org/devel/generated/statsmodels.tsa.exponential_smoothing.ets.ETSModel.html

开始借助 AI 编写或生成代码。

开始借助 AI 编写或生成代码。

开始借助 AI 编写或生成代码。

开始借助 AI 编写或生成代码。

Assignment Week 8 and 9.

Bitcoin is a cryptocurrency. Its price has increased dramatically in the past five years. The BTC-USD.csv contains Bitcoin close prices from 30/04/2013 to 30/04/2018 in column "Close." Please use this data set to answer the following questions:

(1) If you are asked to use autoregressive models to generate forecasting, which variable do you think will work better as dependent variable or variable processing. Why? (20 pts)

- (a) The original value of Bitcoin prices $Close_t$, or
- (b) The first order differenced value $D_Close_t = Close_t - Close_{t-1}$.

(2) Please use an ETS model to forecast for Bitcoin Prices in the next five trading days (keep the answers to two decimal places) (20 points).

(3) Please use autoregressive methods to forecast for Bitcoin Prices in the next five trading days (keep the answers to two decimal places) (20 points).

(4) Please design time-lag features for time-series predictions. https://scikit-learn.org/stable/auto_examples/applications/plot_time_series_lagged_features.html

(5) Design a rolling-forecasting evaluation method to evaluate the "next-day" prediction performance. Please illustrate the validation dataset, evaluation metrics and calculation, and model comparison in your method. (This is a method design question; NO programming is needed.) (20 points)

开始借助 AI 编写或生成代码。

(1) To determine which variable is better for forecasting Bitcoin prices using autoregressive models, let's examine the two options:

Option (a): The Original Bitcoin Prices (Close)

1. **Non-Stationarity:** The original Bitcoin prices are likely to be non-stationary because of trends and price fluctuations. Non-stationary data can lead to unreliable results in forecasting.

2. **Autocorrelation:** While original prices may show some autocorrelation, the trends can complicate the modeling process. The model might need extra components to deal with these trends.

Option (b): The First Order Differenced Value (D_Close)

1. **Stationarity:** The first order differenced value (D_Close) is generally more stationary than the original prices. Differencing helps remove trends and seasonality, making it more suitable for autoregressive models.
2. **Simplicity:** Using differenced values simplifies the modeling process, as there is no need to add extra terms to account for non-stationarity.
3. **Focus on Changes:** Differenced values emphasize price changes rather than absolute levels, which is important for predicting movements in a volatile market like cryptocurrencies.

Conclusion

Considering these points, **Option (b)**, the first order differenced value ($D_{\text{Close}} = \text{Close}_t - \text{Close}_{(t-1)}$), is likely the better choice for the dependent variable in autoregressive models. This is mainly because it tends to be more stationary, which is essential for effective time series forecasting.

(2) Please use an ETS model to forecast for Bitcoin Prices in the next five trading days (keep the answers to two decimal places) (20 points).

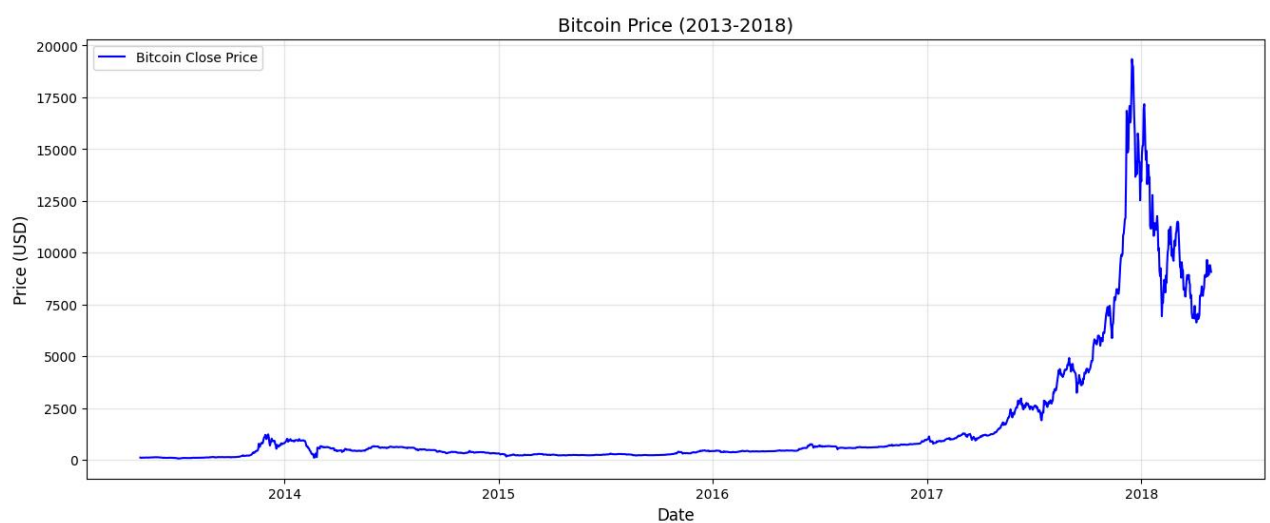
```
import pandas as pd
import numpy as np
from statsmodels.tsa.exponential_smoothing.ets import ETSModel
from matplotlib import pyplot as plt

# Load data
df = pd.read_csv('Q5BTC-USD.csv')

# Convert 'Date' to datetime and set as index
df['Date'] = pd.to_datetime(df['Date'], format='%m/%d/%Y') # Match the date format in the CSV
df = df.set_index('Date')

# Extract the 'Close' price series (ensure data is sorted by date)
btc = df['Close'].sort_index() # Sort by date to maintain time order
```

```
plt.figure(figsize=(16, 6))
plt.plot(btc.index, btc, 'b-', label='Bitcoin Close Price')
plt.title('Bitcoin Price (2013-2018)', fontsize=14)
plt.xlabel('Date', fontsize=12)
plt.ylabel('Price (USD)', fontsize=12)
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



```
model = ETSModel(
    btc,
    error='add',
    trend='add',
```

```

seasonal=None
)

model_fit = model.fit()

forecast_steps = 5
forecast = model_fit.forecast(steps=forecast_steps)

forecast_rounded = np.round(forecast, 2)

# Get the dates for the forecast period
last_date = btc.index[-1]
forecast_dates = pd.date_range(start=last_date, periods=forecast_steps + 1, freq='D')[1:] # Add 1 day to the last date

# Output the forecast results with dates
print(f"Bitcoin Price Forecast (Next {forecast_steps} Trading Days):")
for i in range(forecast_steps):
    print(f"{forecast_dates[i].strftime('%Y-%m-%d')}: ${forecast_rounded.iloc[i]}")

```

```

Bitcoin Price Forecast (Next 5 Trading Days):
2018-05-01: $9082.22
2018-05-02: $9087.14
2018-05-03: $9092.06
2018-05-04: $9096.98
2018-05-05: $9101.9
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provided, but it has
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:837: ValueWarning: No supported index is available. Prediction
return get_prediction_index(
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:837: FutureWarning: No supported index is available. In the ne
return get_prediction_index(

```

(3) Please use autoregressive methods to forecast for Bitcoin Prices in the next five trading days (keep the answers to two decimal places) (20 points).

```

import pandas as pd
import numpy as np
from statsmodels.tsa.ar_model import AutoReg
from statsmodels.graphics.tsaplots import plot_acf
import matplotlib.pyplot as plt

# Load and preprocess data
btc = pd.read_csv('Q5BTC-USD.csv')
btc['Date'] = pd.to_datetime(btc['Date'], format='%m/%d/%Y') # Parse dates
btc = btc.set_index('Date').sort_index() # Set date as index
close_prices = btc['Close'].dropna() # Extract closing prices

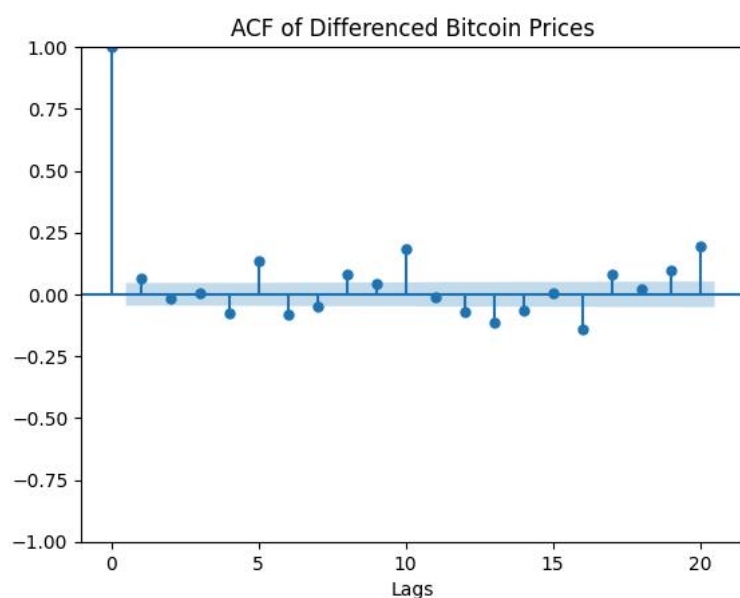
# First-order differencing to make data stationary: D_Close_t = Close_t - Close_{t-1}
diff_close = close_prices.diff().dropna() # Remove NaN from first difference

```

```

# Plot ACF to find significant lags
plot_acf(diff_close, lags=20)
plt.xlabel('Lags')
plt.title('ACF of Differenced Bitcoin Prices')
plt.show()

```



```
# Train AR(1) model on differenced data
p = 1 # Model order
ar_model = AutoReg(diff_close, lags=p)
ar_fit = ar_model.fit()
```

```
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provided, but it has
self._init_dates(dates, freq)
```

```
# Forecast next 5 differenced values
forecast_diff = ar_fit.forecast(steps=5)
```

```
# Recover original price forecasts
last_actual_price = close_prices.iloc[-1] # Last known price
forecast_prices = []
current_price = last_actual_price
```

```
for diff in forecast_diff:
    current_price += diff
    forecast_prices.append(round(current_price, 2)) # Round to 2 decimals
```

```
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:837: ValueWarning: No supported index is available. Prediction
return get_prediction_index(
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:837: FutureWarning: No supported index is available. In the ne
return get_prediction_index(
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/deterministic.py:308: UserWarning: Only PeriodIndexes, DatetimeIndexes with a fr
fcst_index = self._extend_index(index, steps, forecast_index)
```

```
# Generate future dates (assuming 5 consecutive trading days)
last_date = close_prices.index[-1]
future_dates = pd.date_range(start=last_date + pd.Timedelta(days=1), periods=5, freq='B') # 'B' = business days
```

```
# Print forecast
print("Bitcoin Price Forecast (Next 5 Trading Days):")
for date, price in zip(future_dates, forecast_prices):
    print(f"{date.strftime('%Y-%m-%d')}: ${price}")
```

```
Bitcoin Price Forecast (Next 5 Trading Days):
2018-05-01: $9070.61
2018-05-02: $9074.76
2018-05-03: $9079.62
2018-05-04: $9084.53
2018-05-07: $9089.43
```

(4) Please design time-lag features for time-series predictions. https://scikit-learn.org/stable/auto_examples/applications/plot_time_series_lagged_features.html

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from collections import deque

# 1. Load and preprocess data
btc = pd.read_csv('Q5BTC-USD.csv')
btc['Date'] = pd.to_datetime(btc['Date'], format='%m/%d/%Y') # Parse dates
btc = btc.set_index('Date').sort_index() # Sort by date
close_prices = btc['Close'].dropna().to_frame() # Target variable: Close price
close_prices.columns = ['Target'] # Rename for clarity

# 2. Create time-lag features
def create_lagged_features(series, lags):
    """
    Create lagged features (Lag_1 to Lag_p) from a time series.
    series: DataFrame with target variable
    lags: list of lag values (e.g., [1, 2, 3] for Lag_1, Lag_2, Lag_3)
    """
    df = series.copy()
    for lag in lags:
        df[f'Lag_{lag}'] = df['Target'].shift(lag) # Shift backward by 'lag' steps
    df = df.dropna() # Remove rows with NaN (from shifting)
    return df

# Define lags (e.g., use past 3, 7, and 14 days' prices as features)
lags = [1, 3, 7, 14] # Adjust based on domain knowledge (e.g., weekly patterns)
lagged_df = create_lagged_features(close_prices, lags)

# 3. Split into features (X) and target (y)
X = lagged_df.drop('Target', axis=1) # Lagged features: Lag_1, Lag_3, Lag_7, Lag_14
```

```

y = lagged_df['Target'] # Target: Current price

# 4. Train-test split (time-based, avoid data leakage)
train_size = int(0.8 * len(lagged_df))
X_train, X_test = X.iloc[:train_size], X.iloc[train_size:]
y_train, y_test = y.iloc[:train_size], y.iloc[train_size:]

# 5. Train a model (e.g., linear regression)
model = LinearRegression()
model.fit(X_train, y_train)

# 6. Evaluate on test data
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print(f"Test MSE: {mse:.2f}")

# 7. Forecast next 5 trading days
def forecast_future(model, history, lags, steps=5):
    """
    Forecast future values using lagged features.
    history: list or deque containing the most recent observations needed for initial lags.
    lags: list of lag values
    steps: Number of days to forecast
    """
    forecasts = []
    # Use a deque for efficient appending and popping
    current_history = deque(history, maxlen=max(lags))

    for _ in range(steps):
        # Create lagged features from the current history
        # Need to extract elements from the deque based on the lags
        lag_features = [current_history[-lag] if lag <= len(current_history) else history[len(history)-lag] for lag in lags]

        # Predict the next value
        pred = model.predict([lag_features])[0]
        forecasts.append(round(pred, 2))
        # Update the history with the new prediction
        current_history.append(pred)

    return forecasts

# Get the history needed for initial lags (max(lags) observations) as a list
history_list = close_prices['Target'].iloc[-max(lags):].tolist()

# Forecast the next 5 trading days
future_forecasts = forecast_future(model, history_list, lags, steps=5)

# 8. Output future forecasts with dates
last_date = close_prices.index[-1]
future_dates = pd.date_range(start=last_date + pd.Timedelta(days=1), periods=5, freq='B') # Business days
print("\nBitcoin Price Forecast (Next 5 Trading Days):")
for date, price in zip(future_dates, future_forecasts):
    print(f"{date.strftime('%Y-%m-%d')}: ${price}")

```

Test MSE: 274370.82

Bitcoin Price Forecast (Next 5 Trading Days):

2018-05-01: \$9100.82

2018-05-02: \$9079.38

2018-05-03: \$9063.66

2018-05-04: \$9023.93

2018-05-07: \$8979.92

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but LinearRegression will attempt to use them anyway

warnings.warn(

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but LinearRegression will attempt to use them anyway

warnings.warn(

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but LinearRegression will attempt to use them anyway

warnings.warn(

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but LinearRegression will attempt to use them anyway

warnings.warn(

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but LinearRegression will attempt to use them anyway

warnings.warn(

(5) Design a rolling-forecasting evaluation method to evaluate the “next-day” prediction performance. Please illustrate the validation dataset, evaluation metrics and calculation, and model comparison in your method. (This is a method design question; NO programming is needed.) (20 points)

To design a rolling-forecasting evaluation method for predicting the next day’s Bitcoin prices, we can follow these steps:

1. Validation Dataset

- **Data Split:** First, divide the dataset into two parts: a training set and a test set. For example, you could use data from 30/04/2013 to 30/04/2017 for training and data from 01/05/2017 to 30/04/2018 for testing.
- **Rolling Window:** Use a rolling window approach. Start with a small training set and gradually add one day at a time. Each time, you will predict the closing price for the next day.

2. Forecasting Process

- **Model Training:** At each step, train the model using the current training dataset.
- **Next-Day Prediction:** After training, predict the closing price for the next day.
- **Update Dataset:** Add the predicted value to the training set and move the window forward by one day.

3. Evaluation Metrics

To evaluate the prediction performance, use the following metrics:

- **Mean Absolute Error (MAE):**
$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$
 Here, (y_i) is the actual price and $(\hat{y}_i)$ is the predicted price.
- **Root Mean Square Error (RMSE):**
$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$
- **Mean Absolute Percentage Error (MAPE):**
$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

4. Model Comparison

- **Multiple Models:** Implement different autoregressive models (like ARIMA and others) for comparison.
- **Performance Evaluation:** Calculate the evaluation metrics (MAE, RMSE, MAPE) for each model using the predictions for the entire test set.
- **Comparison Table:** Create a table to summarize the performance of each model, helping to see which one performs best for next-day predictions.

5. Visualization (Optional)

If you want, you can visualize the actual vs. predicted prices on a graph for each model to make comparisons clearer.

Conclusion

This rolling-forecasting evaluation method provides a structured way to assess the prediction performance of different models for Bitcoin prices. By using an expanding training set and evaluating with clear metrics, you can identify the best model for predicting next-day prices.